MOVIE RECOMMENDATION SYSTEM

Submitted By

Aimen Atif **20-CP-26**
Shaher Bano Sani **20-CP-50**

Submitted To

Dr. Waqar Ahamd

DEPARTMENT OF COMPUTER ENGINEERING
UNIVERSITY OF ENGINEERING AND TECHNOLOGY TAXILA

SEMESTER PROJECT REPORT

TITLE: MOVIE RECOMMENDATION SYSTEM

1. INTRODUCTION

In the modern digital age, the film industry has experienced remarkable expansion, resulting in an abundance of available content and user-generated reviews. This saturation of movies makes it increasingly difficult for viewers to navigate and make informed choices about what to watch. Movie recommendation systems have become essential in assisting viewers by offering personalized recommendations. These systems significantly enhance our movie recommendation system, ensuring user satisfaction by providing tailored suggestions that align with their preferences.

2. PROBLEM STATEMENT

The rapid expansion of movie content has resulted in an overwhelming amount of information, posing a challenge for movie enthusiasts seeking personalized recommendations based on their preferences. Constructing a movie recommendation system using machine learning techniques can encounter obstacles in effectively managing large datasets. While machine learning approaches excel at delivering precise recommendations based on user behavior and preferences, there is a growing concern regarding their scalability to handle the continuously growing data volume.

3. OBJECTIVES

The primary objective of this project is to create a content-based movie recommendation system that delivers personalized movie suggestions to users. By examining the content of movies and taking into account user preferences, the system strives to generate recommendations that closely match each user's unique tastes and interests. This focus on personalization enhances the overall user experience and significantly improves the chances of users discovering movies that align with their preferences.

4. TOOLS & LIBRARIES

Tools:

- VS Code
- Jupyter

Libraries:

- Pandas
- Scikit-learn.
- difflib
- Pickle

5. EXPLANATION

Content-Based Filtering:

Content-based recommendation systems analyze the similarity between different users or items to provide recommendations. These systems examine the attributes and descriptions of items to identify common characteristics among a user's favorite items, which are then stored in the user's profile. In a specific case, a content-based recommender utilized cosine similarity as a prediction method. It

considered various attributes such as keywords, taglines, director, and cast from the sources. A general flow diagram of content-based recommendation system is given in Figure 1.1.

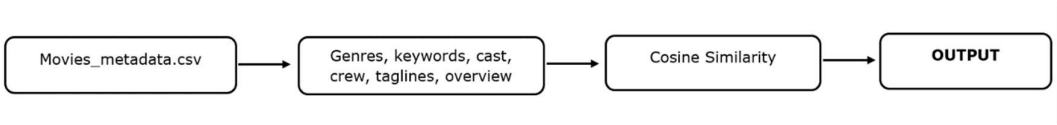


Figure 1.1 Content-based Recommendation system

In the case of content-based recommendation systems, the similarity between the two items is calculated by means of Cosine Similarity Function. It is a similarity metric commonly used in content-based recommendation systems. It measures the similarity between two items by calculating the cosine of the angle between their feature vectors. In the case of Movie Recommendation Systems, the feature vector represents the attributes or content of the movies. The cosine similarity value ranges from 0 to 1, where 0 indicates no similarity, and 1 represents perfect similarity. Equation 1.1 shows the Cosine Similarity between two items x and y is given as:

$$\frac{\sum x_i y_i}{\sqrt{\sum x_i^2}\sqrt{\sum y_i^2}} \qquad (1.2)$$

To utilize cosine similarity in a content-based recommendation system, the system first constructs feature vectors for each item using relevant attributes. Then, when a user expresses their preference for a specific item, the system calculates the cosine similarity between that item and other items in the dataset. The items with the highest cosine similarity scores are considered most similar to the user's preferred item and are recommended accordingly. Figure 1.2 illustrates how the cosine similarity technique is used to calculate the similarity vector between two movies M1 and M2.

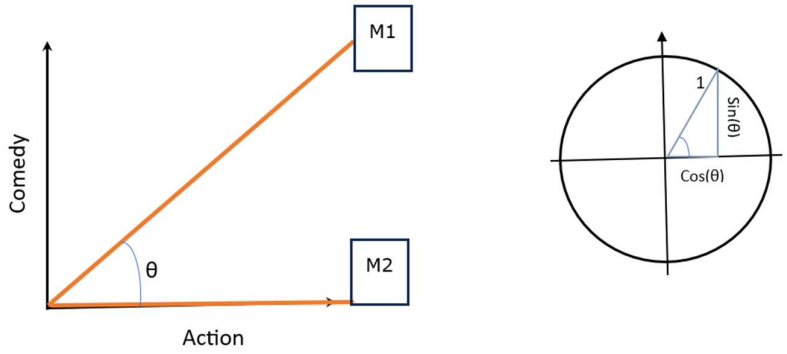


Figure 1.2 Cosine Similarity

6. METHODOLOGY:

The content-based movie recommendation system makes the use of movie metadata and generate recommendations based on genres, and overviews. The steps in our suggested methodology are as under:

1. Data Inspection
2. Selection of Attributes
3. Text Vectorization
4. Cosine Similarity
5. Recommendation of movies
6. Serialization

I. Data Inspection

This step consists of the detailed exploration of the dataset of movies metadata. By means of different commands from the Python environment, we can check and determine, how the actual dataset looks like. This is done through various strategies including visualizing the whole dataset or by looking for different types of uncertainties that may occur in it so that we can remove them in initial steps. Firstly, the dataset file is loaded into python and then by using various commands, it is inspected.

- **film.head** command here is used to comprehend the attributes of the dataset, their values, and datatypes. We have used it to display the initial ten movies including their ID, Title, genre, collection, overview, popularity, original language, release date, runtime, vote average, and vote counts from the dataset. The purpose behind using this is to quickly get a glimpse of how the data looks like.
- **film.describe** function is used to manipulate the tabular dataset to generate its descriptive statistics. By using it we have displayed the count, mean, standard deviation, minimum and maximum in terms of ID, popularity, vote average and vote count. These statistics can give the idea of the overall distribution is valuable for understanding the characteristics of the data in designing and implementing the recommendation algorithm.
- **film.info** is used to illustrate the range index of the dataset file, by displaying the total number of entries or rows in it, total columns, count of the entries, non-null entries, and their count in terms of d-types. It also helps to find the total memory usage that dataset file is acquiring.
- The dataset is further inspected by using the **film.isnull( ).sum( )** command which basically displays the number of null values within all of the attributes or columns of the file. The primary purpose of using it is to check whether there exists any empty or invalid value within the dataset so that we can apply any kind of preprocessing to cope up with it.

II. Selection of Attributes

In this part, we basically have to extract those columns which will be used in the designing of the recommendation system. For example, the dataset includes the attributes of ID, Title, genre, collection, overview, popularity, original language, release date, runtime, vote average, and vote counts. But the system has to recommend movies based on the genres and overviews of the searched movie, and the result will be in the form of movie ID and their respective titles. The genres give the details of characteristics, themes, and styles of the movie, while the overview refers to a summary of the respective movie that provides the general interpretation of movie's storyline, main characteristics and key elements. Both attributes are combined to provide comprehensive and enhanced recommendations. So, we only have to use the features or attributes of ID, title, genres and overviews.

- Firstly, we have displayed the columns of dataset by using the **film.columns** command. It gives information of the data types of all the attributes along with them as well.
- Then, we extracted the ID, title, genre, and overview columns from the dataset, that will be used to recommend the movies. And from these, the genre and overview attributes are combined together under the tag column. This single column is used for the conversion and extraction purpose in the next steps.

III. Text Vectorization

The process of transforming textual data into a numerical representation suitable for analysis and machine learning applications is known as text vectorization. It involves turning unstructured text materials into a format that is organized and readable by algorithms. These vectors are then utilized in calculating the similarities between the movies for the recommendation purpose.

- For the feature extraction purpose, we have used sklearn library, to import the TF-IDF Vectorizer.

- It is basically used to convert the textual data that is the combination of genre and overview attribute into the numeric representation. These numeric feature matrices are used by the system for further processing. The maximum rows of the dataset and the stop words language, both are defined. According to the number of rows, the dimensions of the vector have resulted in 45000x45000.
- Lastly, the object vector fits into the text data. This step analyzes the text and builds the vocabulary (a set of unique words or terms) based on the 'tags' column of the new_data data frame. It learns the vocabulary and assigns a unique index to each word. The **.toarray()** method is then used to convert the resulting sparse matrix representation into a dense array format.

IV. Applying Cosine Similarity

Cosine similarity technique is used here to calculate the similarity between different movies based on their genres and overviews that were previously combined into a single attribute. It measures the similarity by calculating the cosine of the angle between their feature vectors. Its value ranges from 0 to 1, where 0 indicates no similarity, and 1 represents perfect similarity. The array that will be formed as a result of this will be used to recommend the movies for the searched ones. The movies with the highest cosine similarity scores are considered most similar to the user's preferred ones and are recommended accordingly.

- Firstly, we have imported the cosine similarity function from the sklearn metrics pairwise library.
- Then, the vector which was created in the text vectorization step is fed into the function and the similarity is calculated. These similarity values are then used to predict the most similar movies in the form of recommendations.

V. Recommendation

This step is used to create a python function for the recommendations. The similarity values from the above step are utilized in recommending movies to the users. The function first finds the index of the given movie title in the Data Frame. Then, it retrieves the similarity scores for that movie from the similarity matrix. The similarity scores are sorted in descending order, excluding the first entry (which corresponds to the movie itself). The top N similar movies are selected based on the sorted similarity scores. Finally, the function retrieves the titles of the top similar movies and prints them as recommendations for the given movie title.

- In the first step, the title column of the dataset is accessed by the function so that it can be used to match the titles of movies for analysis.
- For the processing purpose, a movie title is given to the function in the first step, it finds the index number of it from the whole dataset. This creates a Boolean mask by comparing each element in the 'title' column with the string of the given movie title. The result is a series of True and False values, where True indicates a match.
- Then the whole data frame is filtered to keep only the rows where the 'title' column matches the given movie title. And afterwards, accesses the index of the given movie from the filtered data frame. The. The **index** attribute returns the index labels in the result.
- The similarity scores of the movies are sorted in descending order, starting from the maximum similarity score to movie to least similar. The **key=lambda vector: vector [1]** command specifies that the sorting should be based on the second element of each tuple, which is the similarity score. The reason is to recommend movies that are highly similar. A tuple is created containing the index of the given movie and its similarity score using the **enumerate(similarity)** function.

- Then, a loop is created to display the first few tuples from the distance list created. The reason behind this is to only show the titles of the top recommended movies. According to the requirement of the recommendation system, we can change the number of movies to be displayed as well as having only displayed first five movies here.

VI. Data Serialization

Data serialization is the process of converting complex data structures, such as objects or data collections, into a format that can be stored or transmitted. We have used the pickle library from the python environment to serialize the calculations done above into the binary format to store them in the form of different files. This step consists of creating two files from the above system that are the movie list and similarity files.

- Movie list file consists of the new_data that includes the movie ID, title and their genres and overviews combined together.
- Similarity file includes the similarity scores of all the movies in the new_data.

Both files are used in the recommendation system to recommend the movies to the users based on the one they have searched for.

7. CODE

- **BACK-END**

```
#IMPORTING LIBRARIES
import numpy as np
import pandas as pd
import difflib
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

#DATA READING
movies_data=pd.read_csv('movies.csv')
movies_data

#DATA INSPECTION
movies_data.head(2)
movies_data.columns
movies_data.describe()
movies_data.info()

#FEATURE SELECTION
selected_features=['title','genres','overview','keywords','tagline','cast','director']
print(selected_features)

#replacing  the null values with null string
for feature in selected_features:
    movies_data[feature]=movies_data[feature].fillna('')

combine_features=movies_data['genres']+'                    '+movies_data['keywords']+'
'+movies_data['tagline']+' '+movies_data['cast']+' '+movies_data['director']
```

```
movies_data.iloc[0]['keywords']
movies_data.iloc[0]['cast']
print(combine_features)

#TEXT VECTORIZATION
from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer=TfidfVectorizer()

feature_vectors=vectorizer.fit_transform(combine_features)
print(feature_vectors)

#COSINE SIMILARITY
similarity=cosine_similarity(feature_vectors)
print(similarity)
similarity.shape

#GETTING USER INPUT FOR MOVIE
movie_name=input('Enter your favourite movie Name: ')


#creating the list with all the movie names given in the data_set
list_of_all_titles=movies_data['title'].tolist()
print(list_of_all_titles)

#finding the close  match for the movie name given by the user
find_close_match=difflib.get_close_matches(movie_name,list_of_all_titles)
print(find_close_match)

close_match=find_close_match[0]
print(close_match)

#finding the index from the movie with title

index_of_the_movie=movies_data[movies_data.title==close_match]['index'].values[0]
print(index_of_the_movie)

#GETTING THE SIMILARITY FOR THE MOVIES
similarity_score=list(enumerate(similarity[index_of_the_movie]))
print(similarity_score)
len(similarity_score)

#SORTING THE MOVIES BASED ON THEIR SIMILARITY SCORE
sorted_similar_movies=sorted(similarity_score,key=lambda x:x[1],reverse=True)
print(sorted_similar_movies)

#MOVIE RECOMMENDATION SYSTEM
print('HYY!! WELCOME TO RECOMMENDATION SYSTEM \n')
movie_name=input('Enter the name of the Movie you want to watch: ')
```

```
list_of_all_titles=movies_data['title'].tolist()

find_close_match=difflib.get_close_matches(movie_name,list_of_all_titles)
close_match=find_close_match[0]

index_of_the_movie=movies_data[movies_data.title==close_match]['index'].values[0]
similarity_score=list(enumerate(similarity[index_of_the_movie]))
sorted_similar_movies=sorted(similarity_score,key=lambda x:x[1],reverse=True)
print('MOVIE SUGGESTED FOR YOU ARE : \n')
i=1
for movie in sorted_similar_movies:
    index=movie[0]
    title_from_index=movies_data[movies_data.index==index]['title'].values[0]
    if (i<10):
        print(i,'.',title_from_index)
        i+=1

#SERIALIZATION
import pickle
pickle.dump(movies_data, open('movies_list.pkl', 'wb'))
pickle.dump(sorted_similar_movies, open('similarity.pkl', 'wb'))
pickle.load(open('movies_list.pkl', 'rb'))
```

- **FRONT-END**

```
import streamlit as st
import pickle
import requests

def fetch_poster(movie_id):
                                                        url                              =
"https://api.themoviedb.org/3/movie/{}?api_key=c7ec19ffdd3279641fb606d19ceb9bb1&language
=en-US".format(movie_id)
     data=requests.get(url)
     data=data.json()
     poster_path = data['poster_path']
     full_path = "https://image.tmdb.org/t/p/w500/"+poster_path
     return full_path

movies = pickle.load(open("movies_list.pkl", 'rb'))
similarity = pickle.load(open("similarity.pkl", 'rb'))
movies_list=movies['title'].values

st.header("Movie Recommender System")

import streamlit.components.v1 as components
```

```
imageCarouselComponent     =     components.declare_component("image-carousel-component",
path="frontend/public")


imageUrls = [
    fetch_poster(1632),
    fetch_poster(299536),
    fetch_poster(17455),
    fetch_poster(2830),
    fetch_poster(429422),
    fetch_poster(9722),
    fetch_poster(13972),
    fetch_poster(240),
    fetch_poster(155),
    fetch_poster(598),
    fetch_poster(914),
    fetch_poster(255709),
    fetch_poster(572154)

    ]
imageCarouselComponent(imageUrls=imageUrls, height=200)
selectvalue=st.selectbox("Select movie from dropdown", movies_list)

def recommend(movie):
    index=movies[movies['title']==movie].index[0]
      distance  =  sorted(list(enumerate(similarity[index])),  reverse=True,  key=lambda
vector:vector[1])
    recommend_movie=[]
    recommend_poster=[]
    for i in distance[1:6]:
        movies_id=movies.iloc[i[0]].id
        recommend_movie.append(movies.iloc[i[0]].title)
        recommend_poster.append(fetch_poster(movies_id))
    return recommend_movie, recommend_poster
if st.button("Show Recommend"):

    movie_name, movie_poster = recommend(selectvalue)
    col1,col2,col3,col4,col5=st.columns(5)
    with col1:
        st.text(movie_name[0])
        st.image(movie_poster[0])
    with col2:
        st.text(movie_name[1])
        st.image(movie_poster[1])
    with col3:
        st.text(movie_name[2])
        st.image(movie_poster[2])
    with col4:
        st.text(movie_name[3])
```

```
        st.image(movie_poster[3])
    with col5:
        st.text(movie_name[4])
        st.image(movie_poster[4])
```

8. IMPLEMENTATION

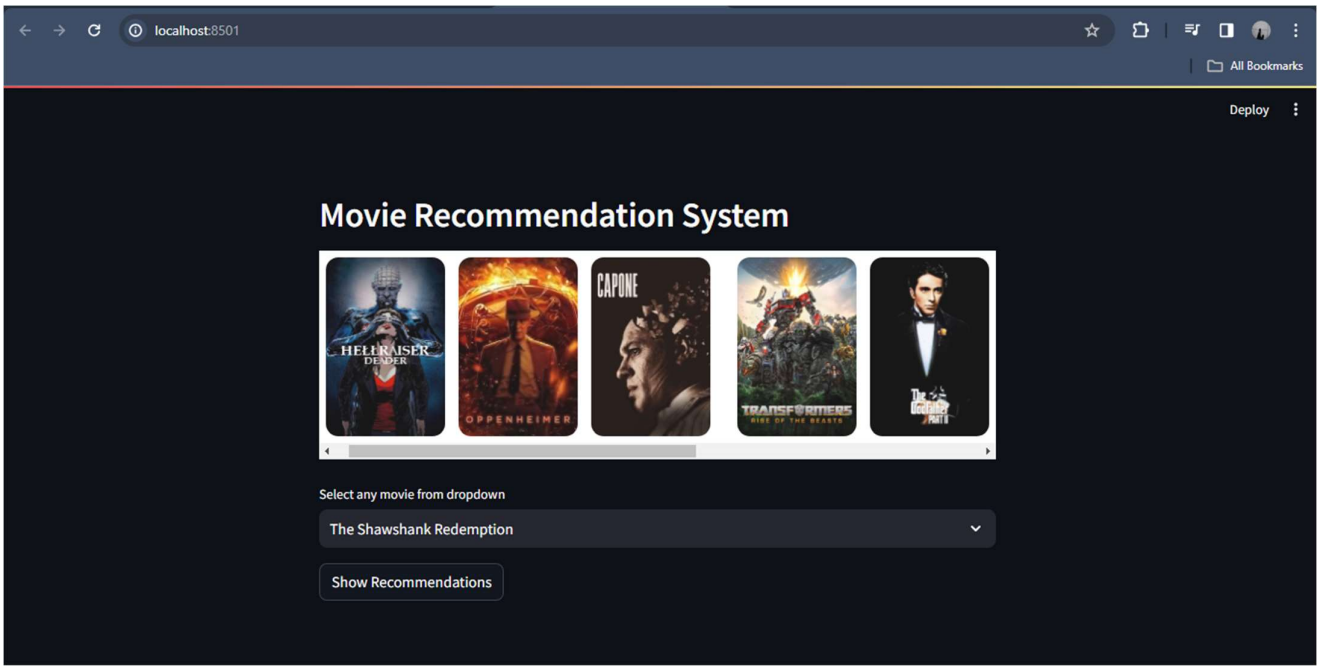
- **BACK-END**

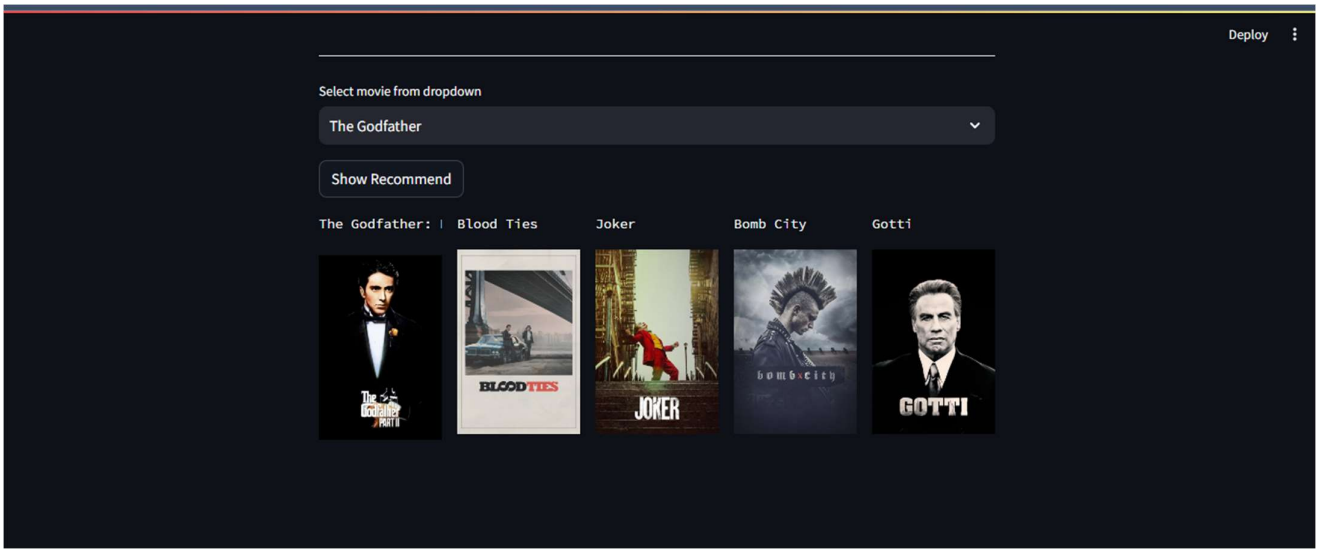
```
HYY!! WELCOME TO RECOMMENDATION SYSTEM
Enter the name of the Movie you want to watch: The Avengers
```

```
MOVIE SUGGESTED FOR YOU ARE :

1 . The Avengers
2 . Avengers: Age of Ultron
3 . Captain America: The Winter Soldier
4 . Captain America: Civil War
5 . Iron Man 2
6 . Thor: The Dark World
7 . X-Men
8 . The Incredible Hulk
9 . X-Men: Apocalypse
10 . Ant-Man
11 . Thor
12 . X2
13 . X-Men: The Last Stand
14 . Deadpool
15 . X-Men: Days of Future Past
16 . Captain America: The First Avenger
17 . The Amazing Spider-Man 2
18 . The Image Revolution
19 . Iron Man
20 . Iron Man 3
21 . Man of Steel
```

- **FRONT-END**





9. CONCLUSION

This project, powered by machine learning techniques, exhibits significant potential in elevating user satisfaction and engagement during the movie-watching process. Its ability to overcome the shortcomings of traditional collaborative filtering systems by prioritizing movie content and individual user preferences is remarkable. By offering personalized recommendations, the system empowers users to venture into unexplored movies and genres, enriching their cinematic experience. Undoubtedly, the content-based movie recommendation system holds immense promise in delivering customized movie suggestions, thereby enhancing the overall movie-watching journey for users.